

Sistemas Informáticos

Instalación, despliegue y funcionamiento de aplicaciones Java en entornos reales

Módulo: Sistemas Informáticos

Curso: 1º DAM

Sergio Blázquez

Pablo Hernández

Mohamed Boukoba

Índice

1. Objetivo del trabajo.....	1
2. Contenidos que se deben investigar.....	1
2.1 Entorno de ejecución de aplicaciones Java.....	1
2.1.1 La Máquina Virtual de Java (JVM).....	1
2.1.2 Diferencia entre JDK y JRE.....	1
2.1.3 Instalación de Java en Windows y Linux.....	2
2.1.4 Variables JAVA_HOME y PATH.....	3
2.2 Creación de instaladores para aplicaciones Java.....	3
2.2.1 Qué es un instalador.....	3
2.2.2 Herramientas para generar instaladores Java.....	4
2.2.3 Elementos que puede incluir un instalador.....	4
2.3 Servicios, APIs y comunicación en red.....	5
2.3.1 Qué es una API.....	5
2.3.2 Spring Boot.....	5
2.3.3 Conceptos básicos de red relacionados con una API.....	6
2.4 Contenedores con Docker.....	6
2.4.1 Qué es Docker.....	6
2.4.2 Máquina virtual vs contenedor.....	7
2.4.3 Conceptos básicos de Docker.....	7
2.4.4 Docker y Java: Spring Boot en un contenedor.....	8
2.5 Aplicaciones frontend con Angular y React.....	8
2.5.1 Qué es una aplicación frontend.....	8
2.5.2 Angular y React.....	9
2.5.3 Ejecución: Node.js, npm y servidor de desarrollo.....	9
2.5.4 Build y archivos generados.....	10
2.5.5 Despliegue en un servidor web.....	10
2.5.6 Relación con el backend Java y las APIs.....	10

1. Objetivo del trabajo

El objetivo de este trabajo es investigar cómo una aplicación de Java pasa de ser un programa creado por un programador a funcionar correctamente en un equipo, en una red o en un servidor. En este trabajo vamos a hablar de la parte de Sistemas Informáticos: instalación, configuración, ejecución, servicios, seguridad, despliegue y mantenimiento básico.

Hablaremos de qué elementos del sistema intervienen en la ejecución de una aplicación: el sistema operativo, la máquina virtual de Java, los puertos de comunicación, el servidor, la base de datos, los permisos, el firewall, las variables de entorno y las herramientas de despliegue.

2. Contenidos investigados

2.1 Entorno de ejecución de aplicaciones Java

2.1.1 La Máquina Virtual de Java (JVM)

Para ejecutar aplicaciones de Java, un equipo de programadores necesita tener la Máquina Virtual de Java (JVM, viene de sus siglas en inglés, Java Virtual Machine). La JVM es un componente de software que actúa como una capa intermedia entre el código compilado de Java (bytecode) y el sistema operativo del equipo. Su función principal es interpretar el bytecode y traducirlo a instrucciones nativas que el procesador pueda ejecutar.

La característica más importante de la JVM es que permite el principio “write once, run anywhere” (escribir una vez, ejecutar en cualquier lugar). Esto significa que un programa Java compilado genera un archivo .class con bytecode que es independiente de la plataforma. Existen implementaciones de la JVM para Windows, Linux, macOS y otros sistemas operativos, de modo que el mismo archivo .class puede ejecutarse en cualquiera de ellos sin necesidad de recompilar el código fuente. Cada sistema operativo dispone de su propia versión de la JVM, que se encarga de adaptar las instrucciones genéricas del bytecode a las particularidades del hardware y cada sistema.

2.1.2 Diferencia entre JDK y JRE

El JDK (Java Development Kit) y el JRE (Java Runtime Environment) son cosas distintas que cumplen funciones distintas en el ecosistema Java:

JRE (Java Runtime Environment): es el entorno de ejecución. Incluye la JVM, las bibliotecas de clases estándar de Java y otros componentes necesarios para ejecutar

aplicaciones de Java ya compiladas. Si solo se quiere ejecutar programas Java, el JRE es suficiente. No contiene herramientas de desarrollo como el compilador.

JDK (Java Development Kit): es el kit de desarrollo. Incluye todo lo que contiene el JRE y además herramientas adicionales para desarrollar aplicaciones: el compilador `javac`, el depurador `jdb`, la herramienta `jar`, `javadoc` para generar documentación, entre otras. El JDK es lo que se debe usar para cuando se necesita escribir, compilar y depurar código Java.

2.1.3 Instalación de Java en Windows y Linux

Instalación en Windows: se puede descargar el instalador del JDK desde el sitio oficial de Oracle (oracle.com/java) o utilizar distribuciones alternativas como Adoptium (Eclipse Temurin), Amazon Corretto o Azul Zulu. Después de ejecutar el instalador `.exe`, para comprobar si se ha instalado correctamente, se debe abrir una terminal (`cmd` o `PowerShell`) y ejecutar el comando:

```
java -version
```

Este comando muestra la versión de Java instalada. Si aparece un mensaje de error indicando que el comando no se reconoce, significa que Java no se ha instalado correctamente o que la variable `PATH` no está configurada. También se puede verificar el compilador con:

```
javac -version
```

Instalación en Linux: en distribuciones basadas en Debian/Ubuntu se puede instalar con los siguientes comandos:

```
sudo apt update
```

```
sudo apt install default-jdk
```

En distribuciones basadas en Red Hat/Fedora:

```
sudo dnf install java-17-openjdk-devel
```

La verificación se realiza de la misma forma que en Windows, ejecutando `java -version` y `javac -version` en la terminal.

2.1.4 Variables `JAVA_HOME` y `PATH`

Las variables de entorno `JAVA_HOME` y `PATH` son importantes para que el sistema y otras herramientas puedan localizar la instalación de Java:

`JAVA_HOME`: es una variable que indica la ruta del directorio donde está instalado el JDK. Por ejemplo, en Windows podría ser `C:\Program Files\Java\jdk-21` y en Linux `/usr/lib/jvm/java-21-openjdk-amd64`. Muchas herramientas de desarrollo (Maven, Gradle,

Tomcat, IDEs) utilizan esta variable para encontrar el JDK sin necesidad de que el usuario especifique la ruta manualmente.

PATH: es una variable del sistema operativo que contiene una lista de directorios en los que el sistema busca ejecutables. Al añadir el directorio bin del JDK a la variable PATH (por ejemplo, C:\Program Files\Java\jdk-21\bin en Windows), se pueden ejecutar comandos como java, javac o jar desde cualquier ubicación en la terminal, sin necesidad de escribir la ruta completa.

Si estas variables no están configuradas, pueden aparecer problemas como: el comando java no se reconoce en la terminal; las herramientas de construcción como Maven o Gradle no encuentran el compilador; los servidores de aplicaciones como Tomcat no arrancan; o los entornos de desarrollo integrado (IDE) como IntelliJ o Eclipse no pueden compilar el código. Para configurarlas en Windows, se accede a Propiedades del sistema > Variables de entorno; en Linux, se añaden al archivo ~/.bashrc o ~/.profile con líneas como:

```
export JAVA_HOME=/usr/lib/jvm/java-21-openjdk-amd64
export PATH=$JAVA_HOME/bin:$PATH
```

2.2 Creación de instaladores para aplicaciones Java

2.2.1 Qué es un instalador

Un instalador es un programa que automatiza el proceso de despliegue de una aplicación. Su función es copiar los archivos necesarios al directorio adecuado, configurar el sistema, crear accesos directos, registrar dependencias y ofrecer una opción de desinstalación limpia (esto no todas, pero sí la mayoría).

Entregar simplemente un archivo .jar o un archivo comprimido tiene varias desventajas frente a un instalador: el usuario debe saber dónde ubicar los archivos y cómo ejecutarlos; no se crean accesos directos automáticamente; no se verifica si existe un JRE instalado en el sistema; no se configuran variables de entorno ni asociaciones de archivos; y no existe un mecanismo sencillo para desinstalar la aplicación. Un instalador profesional soluciona todos estos problemas y ofrece una experiencia de usuario mucho más amigable.

2.2.2 Herramientas para generar instaladores Java

Inno Setup: es una herramienta gratuita y muy popular para crear instaladores en Windows. No está diseñada específicamente para Java, pero se puede utilizar para empaquetar archivos .jar junto con un JRE embebido. Permite crear asistentes de instalación personalizados con selección de idioma, licencia, directorio de destino y opciones avanzadas. Su punto fuerte es la sencillez y la amplia documentación disponible.

install4j: es una herramienta comercial multiplataforma específicamente diseñada para aplicaciones Java. Permite generar instaladores para Windows, macOS y Linux desde una única configuración. Ofrece detección automática de JRE, posibilidad de empaquetar un JRE propio, creación de servicios de sistema (daemons), generación de lanzadores nativos y actualizaciones automáticas. Es una opción muy completa para proyectos profesionales.

jpakeage: es la herramienta oficial incluida en el JDK desde la versión 14 y estabilizada en la versión 16. Permite empaquetar aplicaciones Java en instaladores nativos (.msi y .exe en Windows, .dmg y .pkg en macOS, .deb y .rpm en Linux) que incluyen un JRE personalizado. Al ser parte del JDK, no requiere herramientas externas y se integra directamente en el flujo de construcción del proyecto.

Launch4j: es una herramienta gratuita que permite convertir un archivo .jar en un ejecutable .exe de Windows. No crea un instalador completo, sino un lanzador nativo que busca un JRE instalado en el sistema o utiliza uno empaquetado. Se puede combinar con Inno Setup u otra herramienta para crear un instalador completo que incluya el ejecutable generado por Launch4j.

2.2.3 Elementos que puede incluir un instalador

- Archivos de la aplicación: el ejecutable .jar, bibliotecas adicionales, archivos de configuración y recursos (imágenes, idiomas, etc.).
- JRE embebido: una copia del entorno de ejecución de Java para garantizar que la aplicación funcione aunque el usuario no tenga Java instalado.
- Accesos directos: en el escritorio, en el menú de inicio o en la barra de tareas, para facilitar el acceso a la aplicación.
- Configuración inicial: archivos de preferencias, registro de claves, asociaciones de tipos de archivo y variables de entorno específicas.
- Dependencias: bibliotecas nativas (.dll en Windows, .so en Linux), controladores de bases de datos u otros componentes requeridos.
- Desinstalador: permite eliminar completamente la aplicación del sistema, borrando archivos, accesos directos y entradas del registro.
- Rutas de instalación personalizables: el usuario puede elegir el directorio donde se instalará la aplicación, o bien aceptar la ruta por defecto.

2.3 Servicios, APIs y comunicación en red

2.3.1 Qué es una API

Una API (Application Programming Interface, Interfaz de Programación de Aplicaciones) es un conjunto de reglas y protocolos que define cómo se comunican distintos

componentes de software entre sí. En el contexto del modelo cliente-servidor, una API actúa como el contrato que especifica qué peticiones puede hacer el cliente, cómo debe formatearlas y qué respuestas puede esperar del servidor.

En la arquitectura cliente-servidor, el cliente (por ejemplo, una aplicación web o móvil) envía peticiones a través de la API al servidor, que procesa la solicitud, accede a la base de datos o a otros servicios, y devuelve una respuesta. La API desacopla la interfaz de usuario de la lógica de negocio, lo que permite que múltiples clientes (web, móvil, escritorio) compartan el mismo backend. Las APIs REST (Representational State Transfer) son el estándar más extendido actualmente para la comunicación entre aplicaciones a través de HTTP.

2.3.2 Spring Boot

Spring Boot es un framework de Java que simplifica la creación de servicios y APIs. Se construye sobre el ecosistema Spring, pero elimina parte de la configuración manual que este requería, siguiendo el principio de «convención sobre configuración».

Las principales razones por las que Spring Boot se utiliza para crear APIs en Java son: incorpora un servidor web embebido (Tomcat por defecto), lo que permite ejecutar la aplicación como un simple archivo .jar sin necesidad de desplegarla en un servidor de aplicaciones externo; ofrece autoconfiguración, que detecta automáticamente las bibliotecas presentes en el proyecto y configura los beans necesarios, también proporciona starters, que son dependencias que agrupan todas las bibliotecas necesarias para una funcionalidad concreta (por ejemplo, spring-boot-starter-web para crear APIs REST); incluye herramientas de desarrollo como spring-boot-devtools para recarga en caliente; y ofrece Actuator para monitorización y gestión de la aplicación en producción.

2.3.3 Conceptos básicos de red relacionados con una API

Dirección IP: es un número único que identifica a cada dispositivo conectado a una red. Puede ser IPv4 (por ejemplo, 192.168.1.100) o IPv6. Los servidores que alojan APIs tienen una dirección IP que los clientes utilizan para localizarlos. En entornos de desarrollo, se suele usar la dirección localhost (127.0.0.1), que hace referencia al propio equipo.

Puerto: es un número (de 0 a 65535) que identifica un proceso o servicio específico dentro de un equipo. Mientras que la dirección IP localiza el equipo, el puerto localiza la aplicación dentro de ese equipo. Por ejemplo, un servidor Spring Boot suele escuchar en el puerto 8080 por defecto, mientras que un servidor web HTTP convencional usa el puerto 80 y HTTPS el 443.

Protocolo HTTP/HTTPS: HTTP (HyperText Transfer Protocol) es el protocolo de comunicación que utilizan las APIs REST para intercambiar mensajes entre cliente y servidor. Define la estructura de las peticiones y las respuestas. HTTPS es la versión segura

de HTTP, que cifra la comunicación mediante TLS/SSL, protegiendo los datos de interceptaciones. En producción, toda API debe usar HTTPS para garantizar la seguridad de las transmisiones.

Peticiones GET y POST: son dos de los métodos HTTP más utilizados. GET se usa para solicitar datos del servidor sin modificar el estado de la aplicación (por ejemplo, obtener la lista de productos). POST se usa para enviar datos al servidor con el fin de crear un nuevo recurso o procesar información (por ejemplo, registrar un nuevo usuario). Otros métodos comunes son PUT (actualizar un recurso existente), DELETE (eliminar un recurso) y PATCH (actualizar parcialmente un recurso).

Respuesta del servidor: tras procesar una petición, el servidor devuelve una respuesta que incluye un código de estado (200 para éxito, 404 para recurso no encontrado, 500 para error del servidor, etc.), cabeceras con metadatos y, habitualmente, un cuerpo en formato JSON con los datos solicitados o el resultado de la operación.

2.4 Contenedores con Docker

2.4.1 Qué es Docker

Docker es una plataforma de software que permite crear, distribuir y ejecutar aplicaciones dentro de contenedores. Un contenedor es un entorno aislado y ligero que incluye todo lo necesario para que una aplicación funcione: código, bibliotecas, herramientas del sistema y configuración. El problema que Docker intenta resolver es que funcione en todas las máquinas, evitando errores provocados por las diferencias entre los entornos de desarrollo, prueba y producción.

Con Docker, la aplicación y todas sus dependencias se empaquetan en una imagen que se puede ejecutar de forma idéntica en cualquier equipo que tenga Docker instalado, independientemente del sistema operativo que tenga. Esto garantiza la reproducibilidad y portabilidad del entorno de ejecución, elimina conflictos entre versiones de bibliotecas y simplifica los procesos de despliegue.

2.4.2 Máquina virtual vs contenedor

Aunque las máquinas virtuales (VM) y los contenedores sirven para aislar aplicaciones, funcionan de manera muy diferente:

Ventajas de las VM: aislamiento completo, compatibilidad con cualquier sistema operativo y mejor seguridad para cargas de trabajo no confiables. Desventajas: mayor consumo de recursos, arranque lento y mayor complejidad de gestión.

Ventajas de los contenedores: ligereza, rapidez de arranque, facilidad de despliegue y escalado, y ser un ecosistema maduro. Desventajas: menor aislamiento que las VM, ya que

comparten kernel. Tampoco son adecuados para ejecutar sistemas operativos distintos al del host.

2.4.3 Conceptos básicos de Docker

Imagen: es una plantilla de solo lectura que contiene el sistema de archivos, las bibliotecas y la configuración necesarios para ejecutar una aplicación. Las imágenes se construyen a partir de un Dockerfile y se almacenan en registros como Docker Hub. A partir de una misma imagen se pueden crear múltiples contenedores.

Contenedor: es la instancia en ejecución de una imagen. Es un proceso aislado que tiene su propio sistema de archivos, red y espacio de procesos. Se puede crear, iniciar, detener y eliminar fácilmente con comandos de Docker.

Dockerfile: es un archivo de texto que contiene las instrucciones paso a paso para construir una imagen Docker. Cada línea define una capa de la imagen (desde la imagen base, la copia de archivos, la instalación de dependencias, hasta el comando de inicio).

Volúmen: es un mecanismo de Docker para persistir datos más allá del ciclo de vida de un contenedor. Los volúmenes almacenan datos en el sistema de archivos del host, de modo que si el contenedor se elimina, los datos no se pierden. Son esenciales para bases de datos o archivos de registro.

Puerto expuesto: es un puerto del contenedor que se hace accesible desde el host o desde otros contenedores. Se configura al ejecutar el contenedor con la opción `-p`, por ejemplo: `docker run -p 8080:8080 mi-app`, que mapea el puerto 8080 del contenedor al puerto 8080 del host.

2.4.4 Docker y Java: Spring Boot en un contenedor

Para empaquetar una aplicación de Spring Boot en una imagen Docker, primero se genera el archivo `.jar` del proyecto con Maven o Gradle. Luego se crea un Dockerfile como el siguiente:

```
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY target/mi-app.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Este Dockerfile utiliza una imagen base ligera (que suele ser más ligera que la normal) con el JRE de Java 21, copia el archivo `.jar` de la aplicación, expone el puerto 8080 y define el comando de inicio. Para construir la imagen se ejecuta:

```
docker build -t mi-app-spring .
```

Y para ejecutar el contenedor:

```
docker run -p 8080:8080 mi-app-spring
```

De esta forma cualquier equipo con Docker instalado puede ejecutar la aplicación con un único comando, sin necesidad de instalar Java, configurar variables de entorno ni preocuparse por las dependencias del sistema.

2.5 Aplicaciones frontend con Angular y React

2.5.1 Qué es una aplicación frontend

Una aplicación frontend es la parte de un sistema software que se ejecuta en el navegador del usuario y con la que este interactúa directamente. En la arquitectura cliente-servidor, el frontend actúa como el cliente: se encarga de presentar la interfaz de usuario, capturar las acciones del usuario (clics, formularios, navegación) y comunicarse con el backend a través de APIs para enviar y recibir datos.

Las aplicaciones frontend modernas se construyen con frameworks de JavaScript como Angular o React, que permiten crear interfaces dinámicas, reactivas y de una sola página (SPA, Single Page Application). En una SPA, la aplicación se carga una única vez y luego actualiza dinámicamente el contenido sin recargar la página completa, lo que proporciona una experiencia de usuario fluida y rápida.

2.5.2 Angular y React

Angular: es un framework de desarrollo frontend mantenido por Google. Está basado en TypeScript y sigue el patrón de arquitectura MVC (Modelo-Vista-Controlador). Ofrece un ecosistema completo que incluye enrutamiento, gestión de formularios, peticiones HTTP, inyección de dependencias y herramientas de testing. Su enfoque es opinionated (opinado) significa que proporciona una estructura definida para organizar el código, lo que lo hace adecuado para proyectos grandes y muchos equipos.

React: es una biblioteca de JavaScript desarrollada por Meta para construir interfaces de usuario basadas en componentes. A diferencia de Angular, React se centra únicamente en la capa de vista y deja que el desarrollador elija las bibliotecas complementarias para enrutamiento, gestión de estado y otras funcionalidades. Utiliza un DOM virtual para optimizar el renderizado y un modelo de programación declarativa que facilita la construcción de interfaces interactivas. Es muy popular por su flexibilidad y su amplio ecosistema.

2.5.3 Ejecución: Node.js, npm y servidor de desarrollo

Tanto Angular como React requieren Node.js y npm para su desarrollo. Node.js es un entorno de ejecución de JavaScript en el lado del servidor que permite ejecutar herramientas

de construcción fuera del navegador. npm (Node Package Manager) es el gestor de paquetes que permite instalar y gestionar las dependencias del proyecto (bibliotecas, frameworks, herramientas).

Para crear un proyecto nuevo con Angular se utiliza el comando `npx @angular/cli new mi-proyecto`, mientras que con React se puede usar `npx create-react-app mi-proyecto` o `npx create-next-app mi-proyecto`. Una vez creado, el servidor de desarrollo se inicia con `npm start` (React) o `ng serve` (Angular). Este servidor ofrece recarga en caliente (hot reload), de modo que los cambios en el código se reflejan inmediatamente en el navegador sin necesidad de recargar la página manualmente.

2.5.4 Build y archivos generados

El proceso de build (construcción) transforma el código fuente de la aplicación frontend en archivos estáticos optimizados listos para ser servidos por un servidor web. Se ejecuta con `npm run build` (React) o `ng build` (Angular). Durante este proceso se realizan varias operaciones: transpilación de TypeScript/JSX a JavaScript estándar, minificación del código (eliminación de espacios y comentarios), ofuscación para proteger el código fuente, agrupación (bundling) de múltiples archivos en uno o varios paquetes, y optimización de imágenes y otros recursos estáticos.

El resultado se almacena en una carpeta, normalmente `dist/` (Angular) o `build/` (React). Esta carpeta contiene archivos HTML, CSS y JavaScript comprimidos, listos para desplegar.

2.5.5 Despliegue en un servidor web

Para desplegar una aplicación frontend en producción, hay que copiar los archivos generados en el proceso de build a un servidor web como Apache HTTP Server, Nginx o incluso un servidor de archivos estáticos como Amazon S3. Nginx es una opción muy popular por su alto rendimiento y su capacidad para servir archivos estáticos de forma eficiente.

En un despliegue típico con Docker, se puede utilizar una imagen Nginx como base y copiar los archivos de la carpeta `dist/` o `build/` al directorio que Nginx sirve por defecto (`/usr/share/nginx/html/`). También es importante configurar el servidor para que todas las rutas de la SPA redirijan al archivo `index.html`, de modo que el enrutamiento del frontend funcione correctamente cuando el usuario accede directamente a una ruta interna de la aplicación.

2.5.6 Relación con el backend Java y las APIs

El frontend y el backend se comunican a través de APIs REST. El frontend (Angular o React) realiza peticiones HTTP al backend Java (Spring Boot), que procesa las solicitudes,

interactúa con la base de datos y devuelve los datos en formato JSON. El frontend luego renderiza esos datos en la interfaz de usuario.

Esta separación de responsabilidades permite que ambos equipos trabajen de forma independiente, siempre que respeten el contrato definido por la API. Para facilitar la comunicación durante el desarrollo, se suelen utilizar herramientas como Swagger/OpenAPI para documentar la API, y se puede configurar un proxy de desarrollo para evitar problemas de CORS (Cross-Origin Resource Sharing). En producción, es habitual utilizar un servidor Nginx como proxy inverso que sirva los archivos estáticos del frontend y redirija las peticiones de la API al backend Spring Boot, creando de esa forma una arquitectura unificada y segura.